

TD/TP 6 — Une quadrature, plusieurs fonctions

Interface abstraite, `virtual` et choix dynamique

Durée : 2 heures

Compilation : `g++ -std=c++20 -Wall -Wextra -pedantic -fsanitize=address,undefined -g affine.cpp gaussian.c
sine.cpp quadrature.cpp main.cpp -o quadrature`

1. Duplication — 15 min

À FAIRE. Comparer les deux fonctions `midpoint` fournies pour `Affine` et `Gaussian`.

TESTER. Vérifier les résultats sur $[0, 1]$.

COMPRENDRE. Le corps appelle seulement `f(x)`.

JALON 1 — DUPLICATION IDENTIFIÉE

2. Interface et Affine — 20 min

À FAIRE. Créer `ScalarFunction` avec `virtual double operator()(double) const = 0`; et un destructeur virtuel. Faire dériver `Affine` et ajouter `override`.

TESTER. Écrire `evaluate(const ScalarFunction&, double)` et vérifier `Affine{2,1}` en 3.

COMPRENDRE. Une référence de base conserve l'objet réel sans copie.

3. Gaussian — 20 min

À FAIRE. Ajouter la formule $\exp(-x^2/(2\sigma^2))$, avec $\sigma > 0$, puis tester l'appel direct et par référence de base.

4. Abstraction et override — 15 min

TESTER. Essayer d'instancier `ScalarFunction`, puis enlever `const` dans une redéfinition portant `override`. Lire les diagnostics et corriger.

JALON 2 — LE COMPILATEUR VÉRIFIE L'INTERFACE

5. Un seul midpoint — 20 min

À FAIRE. Remplacer les surcharges par `double midpoint(const ScalarFunction&, double, double, std::size_t)`. Le corps ne nomme aucun modèle.

TESTER. Tester l'intégrale de x sur $[0,1]$, une constante et la convergence de la gaussienne.

JALON 3 — QUADRATURE INDÉPENDANTE DU MODÈLE

6. Ajouter Sine — 15 min

À FAIRE. Créer `Sine` avec `std::sin(x)`.

TESTER. Vérifier `midpoint(sine,0,pi,20000)  $\approx$  2`.

COMPRENDRE. L'ajout de `Sine` ne modifie pas `midpoint`.

7. Qui possède l'objet ? — 10 min

COMPRENDRE. Comparer objet local, référence et pointeur brut. Une référence utilise mais ne possède pas. Le code `new/delete` n'est qu'un contre-exemple : la propriété moderne sera étudiée en séance 8.

Bilan — 5 min

Répondre : rôle de `virtual`, `override`, destructeur virtuel ; différence entre référence et valeur ; question ouverte : comment posséder plusieurs objets polymorphes sans écrire `new/delete` ?

À rendre. Sources, tests sous sanitizers et réponses courtes.